

Finding Oreo



Group 2: Kelly Jiang, Terrance Huang, Tyler Dow, Emily Huang

Problem Statement & Background

Background

Oreo is Tyler's tuxedo cat, with a distinctive black and white pattern.

We want to evaluate how different models (Logistic Regression, KNN, CNN) perform on identifying Oreo.

The Goal

Classify whether a given cat image is Oreo or not (binary classification).

Hypothesis

CNN will outperform traditional models such as Logistic Regression and KNN.

Why?

CNN learns hierarchical features and preserves spatial structures from the pixel data, which the other models cannot do.

Dataset

~ 10,000

Non-Oreo Cat Images
Kaggle Cat Dataset

820

Oreo Photos
Personal photographs

80/10/10

Train/Val/Test
Stratified Split

Kaggle Cat Dataset

Contains images of cat with an associated annotation file denoting locations of 9 face points:
left eye, right eye, mouth, left ear 1, left ear 2, left ear 3, right ear 1, right ear 2, right ear 3

Source: kaggle.com/datasets/crawford/cat-dataset/data

Class Imbalance

Significant difference between the number of Oreo photos to the other Kaggle Non-Oreo cat photos.

WeightedRandomSampler:

Assigns higher sampling probability to underrepresented Oreo images in training batches.

Preprocessing

Resize

128 x 128 pixels
for all images

This gives a uniform
input for all models.
Makes everything more
consistent.

Augmentation

Random Flip
(Horizontal)
Rotation (+- 15°)
Brightness (+- 20%)

For CNN. It helps
introduce variation in
the data to prevent
overfitting and improve
generalization.

HOG

Orientations=6
Pixels_per_cell=(16,16)
Cells_per_block=(2,2)
Channel_axis=-1

For Logistic
Regression and
K-nearest neighbors. It
helps to extract the
edges and shapes of
the image.

PCA

95% variance

This reduces the
dimensionality of the
data to speed up the
performance.

StandardScale

r

mean = 0
std = 1

This helps the model
weigh features more
equally and converge
faster.

HOG (Histogram of Oriented Gradients)

Main Idea

Represent an image using edge directions

Steps

1. Preprocess (grayscale, resize)
2. Compute gradients to detect edges and their directions
3. Divide the image into small cells
4. Create histograms of edge orientation for each cell
5. Normalize across larger blocks for consistency
6. Concatenate features into a single feature vector

Input image



Histogram of Oriented Gradients



Logistic Regression

Library

from sklearn.linear_model:
LogisticRegression

Hyperparameters

Solver: saga

Best Maximum Iteration: 15,000

Best C: 0.05

Class Weight: balanced

Hyperparameter Tuning: Validation F1 Scores

Selected best C and max_iter based on highest Macro F1

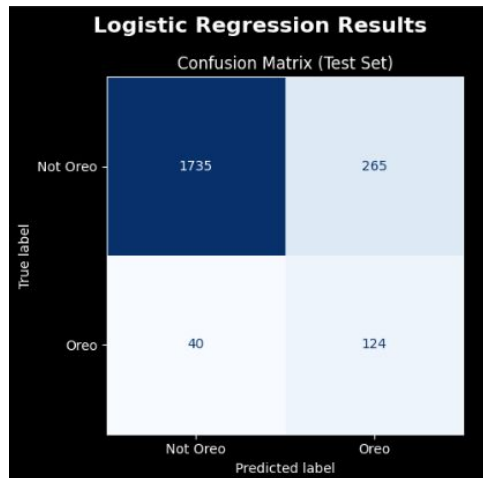
	20.00	10.00	5.00	1.00	0.50	0.10	0.05
15000	0.697765	0.708323	0.701020	0.692168	0.708229	0.709629	0.716242
10000	0.701740	0.701935	0.697009	0.709869	0.684637	0.705683	0.714310
5000	0.684567	0.703897	0.693728	0.702138	0.697009	0.702166	0.714672
1000	0.702313	0.704928	0.706298	0.712355	0.703897	0.695349	0.709869

C = inverse of regularization strength

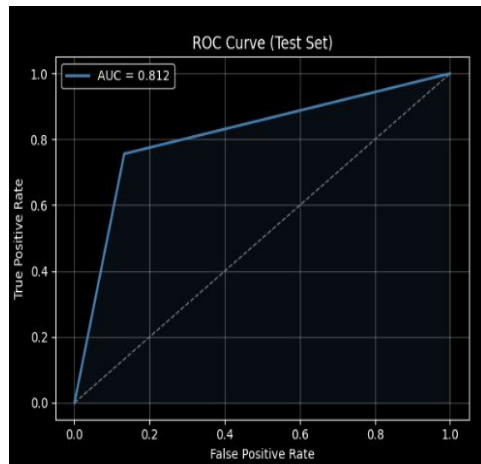
(C = 0.05) means strong regularization - model is penalised heavily for complexity, reducing overfitting on the HOG+PCA feature space.

max_iter = max number of epochs for the solver to converge
(max_iter = 15000)

Logistic Regression Results



Confusion Matrix



ROC-AUC Curve

- The model classifies Non-Oreo photos correctly better than Oreo photos, however, that may be due to the class imbalance
- High false positive number (`class_weight='balanced'`)
- `C=0.05` to prevent overfitting
- Slightly underfitting: too constrained from the `C` value, HOG+PCA, logistic regression in general

Accuracies: 89.16% (Train) 86.73% (Val) 87.25% (Test)

K-nearest Neighbors (KNN)

Library

`sklearn.KNeighborsClassifier`

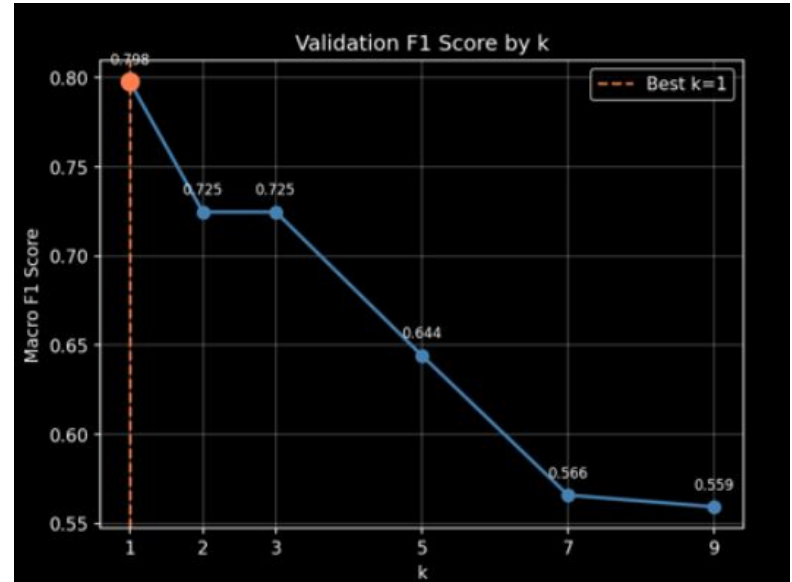
Hyperparameters

Distance formula: Euclidean distance (default)

n_jobs = -1 (uses all processors)

Macro F1 Score for imbalanced dataset

Best k: 1



K-NN Results

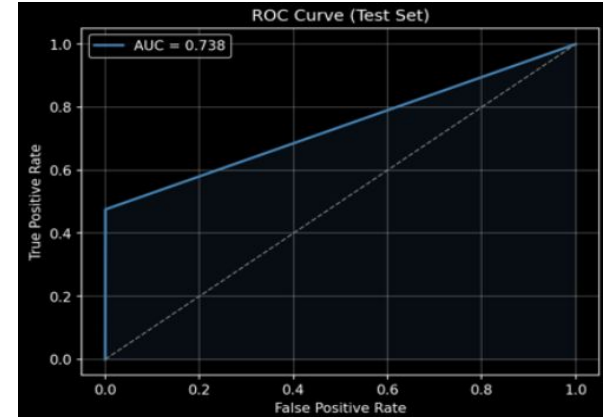
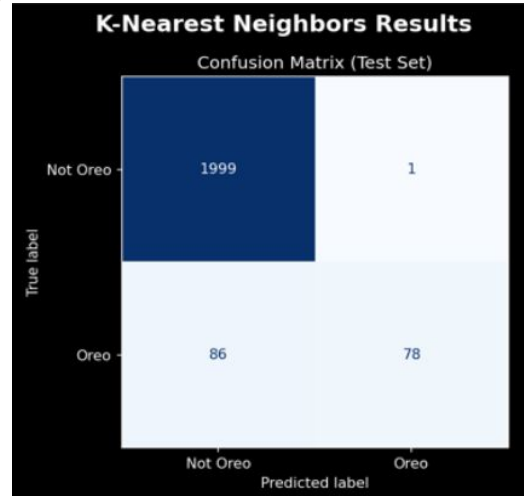
95.98% accuracy

KNN model classifies non-Oreo pictures much better than it does Oreo pictures.

- 1999 true negatives
- 78 true positives
- 1 false positives
- 86 false negatives

The Cause:

Most likely due to class imbalance



ROC curve supports this reasoning: Classifies as Oreo only if confident, otherwise not Oreo to reduce false positives

Convolutional Neural Network (CNN)

Layers

3 full layers
2x2 Pool Size
Leaky ReLU
Dropout Rate Of 0.5

Library

torch.nn

Hyperparameters

Learning Rate: 0.0001

Solver: Adam

Threshold: 0.7

F1: 92.90

Leniency: 5

Hyperparameter Tuning: Validation F1 Scores

	0.10	0.30	0.50	0.70	0.90	0.99
1.00000	0.070354	0.070354	0.480336	0.480336	0.480336	0.480336
0.10000	0.070354	0.075633	0.557046	0.480336	0.501260	0.480336
0.01000	0.849437	0.884772	0.871912	0.893188	0.869698	0.889028
0.00100	0.889470	0.846771	0.909923	0.922396	0.923339	0.903000
0.00010	0.878119	0.915643	0.909310	0.928980	0.916848	0.850577
0.00001	0.818433	0.803506	0.846749	0.890487	0.919347	0.890995

Convolutional Neural Network Evaluation



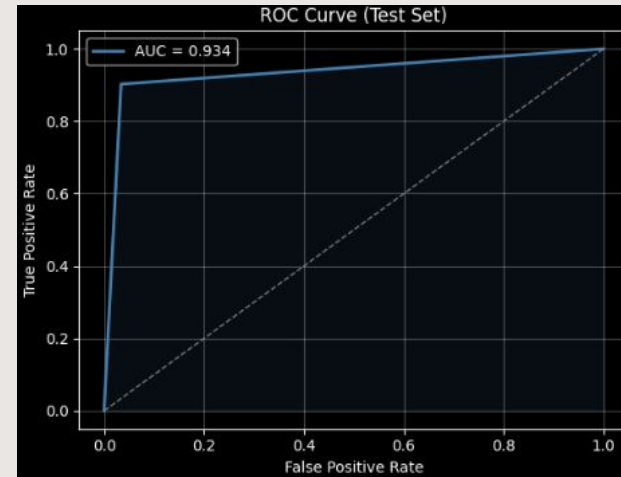
Accuracy: 96.11%

AUC: 93.4%

Problems:

Low Data Sample & Variation

Some Cats Look Similar



Performance

	Logistic Regression	K-Nearest Neighbor	Convolutional Neural Network
Accuracy	0.872458	0.959797	0.961183
Precision	0.350267	0.987342	0.685185
Recall	0.798780	0.475610	0.902439
F1	0.486989	0.641975	0.778947
AUC	0.838640	0.737555	0.934220



Conclusion

Best Performing Model: CNN

CNN learns directly from raw pixel data, preserving spatial relationships between features and builds hierarchical representations. The other models have to have the images compressed to a fixed vector, which loses its spatial information.

Limitations

- Insufficient amount of Oreo pictures compared to Non-Oreo pictures.
- Oreo pictures share similar background and lighting.

Future Work

- Collect more Oreo pictures to reduce class imbalance.
- Tune decision thresholds independently per model for better precision/recall balance.
- Do multi classification with other cats.

References

Dataset: <https://www.kaggle.com/datasets/crawford/cat-dataset/data>

HOG:

<https://www.geeksforgeeks.org/machine-learning/hog-feature-visualization-in-python-using-skimage/>
codegenes.net/blog/hog-pytorch/